Project Executable Files

Date	30 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	3 Marks

Project Executable Files

This section contains all details regarding the project executables, organization, run instructions, and verification checklists.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
Flood-Prediction-main/
|
├── app.py                  # Main Flask application
├── db_setup.py             # Database initialization
├── preprocess.py           # Data preprocessing
├── train.py                # ML model training script
├── requirements.txt        # Python dependencies
├── readme.md               # Project documentation
├── .gitignore              # Git ignore configuration
|
├── data/
|   └── flood_prediction.db     # SQLite database
|
├── models/
|   ├── flood_model.pkl         # Trained ML model
|   ├── scaler.save             # Saved scaler
|   ├── model_meta.json         # Model metadata
|   ├── feature_importance.png  # Feature importance graph
|   └── roc_curve.png           # ROC curve
|
├── static/
|   ├── css/
|   |   └── main.css            # Stylesheet
|   ├── js/
|   |   └── main.js             # JavaScript
|   ├── feature_importance.png
|   └── roc_curve.png
|
└── templates/
    ├── index.html              # Landing page
    ├── home.html               # Home page
    ├── login.html              # Login page
    ├── register.html           # Registration page
    ├── dashboard.html          # User dashboard
    ├── chance.html             # Flood prediction result
```

```
├── no_chance.html          # No flood prediction result
├── prediction_history.html  # Prediction history
├── profile.html             # User profile
├── manage_users.html        # Admin user management
└── model_dashboard.html     # Model analytics dashboard
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://flood-prediction-4rwg.onrender.com/ (Also local: http://127.0.0.1:5001)
Login Credentials (Demo)	Email: **admin@flood.com** Password: **admin**
Platform / Hosting Provider	Render
Repository Link	https://github.com/sameer727/Flood-Prediction
Demo Video Link	https://drive.google.com/file/d/1Zfj0J4-pEeJsZVuYUWqXQb8Ta66CekXI/view?usp=sharing

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- **Python**: version 3.12 or 3.13 installed.
- **Git**: installed (for cloning).
- **Web Browser**: modern browser (Chrome, Edge, Firefox).

Step 1: Clone the repository
Open a shell terminal and run:

```
git clone https://github.com/sameer727/Flood-Prediction.git
cd Flood-Prediction
```

Step 2: Set up Virtual Environment & Install Dependencies
Initialize a clean Python virtual environment and run package installation:

```
# Initialize virtual environment
python -m venv .venv
```

```
# Activate the virtual environment
# On Windows:
.venv\Scripts\activate
# On macOS/Linux:
source .venv/bin/activate

# Install required packages
pip install -r requirements.txt
```

Step 3: Run Database Setup & Seeds

Execute the schema creation script to create tables and set up the default admin user:

```
python db_setup.py
```

Step 4: Tune Database Concurrency

Execute the optimization script to configure WAL mode and index foreign keys:

```
python optimize_db.py
```

Step 5: Start the Application

Boot the web server locally using Python:

```
python app.py
```

Step 6: Open Web Browser

Open your browser and navigate to:
http://127.0.0.1:5001

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	**SQLite Concurrent locking**: Under high virtual user write load, standard SQLite rollback journaling locks the database and queues requests.	**Resolved**: Write-Ahead Logging (WAL) mode enabled and PRAGMA synchronous = NORMAL configured to allow parallel reads and writes.
2	**Single-threaded dev server limits**: Running heavy joins (history query logs) under high concurrency can create queue delays because the Werkzeug dev server is single-threaded.	**Status**: Local development behaves as expected. For production deployment, using a multi-worker WSGI server like Gunicorn or uWSGI is recommended.

3	**Bcrypt authentication CPU load**: High concurrency logins can saturate the CPU due to the resource-intensive hashing computation.	**Status**: Session persistence cookies are used in performance tests to bypass redundant hashing executions.